Code Explanation

Spark Utility Functions

This code provides a set of utility functions for working with Apache Spark, including creating a Spark session, reading and writing data to Oracle tables, creating trigger files, running cleanup scripts, and writing count files.

The utility functions are designed to be reusable and simplify the process of working with Spark and Oracle databases. The functions are well-structured and follow best practices for Spark development.

Overview of the Code

The code consists of six utility functions that can be used to perform various tasks related to Spark and Oracle databases. The functions are designed to be modular and can be used independently of each other.

Step 1: Creating a Spark Session
The `get_spark_session` function creates and returns a Spark session with the specified configuration.

```
return SparkSession.builder
    .appName("E2E_AC_TXDBH_DP_YUYU_CREATION")
    .config("spark.sql.legacy.timeParserPolicy", "LEGACY")
    .config("spark.sql.legacy.parquet.datetimeRebaseModeInWrite", "L
EGACY")
    .getOrCreate()
```

Step 2: Reading Data from Oracle Tables
The `read_oracle_table` function reads data from an Oracle table using JDBC. It takes in a Spark session, connection string, table name, schema name, and an optional query.

```
df = spark.read
    .format("jdbc")
    .option("url", jdbc_url)
    .option("dbtable", f"{schema_name}.{table_name}")
    .option("user", username)
    .option("password", password)
    .option("driver", "oracle.jdbc.driver.OracleDriver")
    .load()
```

Step 3: Writing Data to Oracle Tables
The `write_to_oracle_table` function writes a DataFrame to an Oracle table using JDBC. It takes in a DataFrame, connection string, table name, schema name, and an optional mode.

```
df.write
    .format("jdbc")
    .option("url", jdbc_url)
    .option("dbtable", f"{schema_name}.{table_name}")
    .option("user", username)
    .option("password", password)
    .option("driver", "oracle.jdbc.driver.OracleDriver")
    .mode(mode)
    .save()
```

Step 4: Creating Trigger Files

The `create_trigger_file` function creates or updates a trigger file with the specified content.

```
file_path = os.path.join(output_dir, trigger_file)
with open(file_path, "w") as f:
    f.write(content)
return file_path
```

Step 5: Running Cleanup Scripts
The `run_cleanup_script` function runs a cleanup shell script.

```
script_path = os.path.join(shell_dir, script_name)
try:
    subprocess.run([script_path], check=True)
    return True
except subprocess.CalledProcessError as e:
    print(f"Error running cleanup script: {e}")
    return False
```

Step 6: Writing Count Files
The `write_count_file` function writes the source record count to an output file.

```
count_df = spark.createDataFrame([(count,)], ["SOURCE_RECT"])
output_path = os.path.join(output_dir, output_file)

count_df.coalesce(1).write
    .option("header", "true")
    .mode("overwrite")
    .csv(output_path)

return count
```